

Release 1.0.4 — Product pages + simulated database

The shop moves from inline arrays to a JSON “database”, gains a page per product, and shares one cart across every page. Status: staging / pre-launch.

VERSION 1.0.4 · staging	DATE 2026-10-03	COMMIT 5d9647e · 37 files, +3019 / -365	DEPENDS ON 1.0.2 shop · 1.0.3 photos
-----------------------------------	---------------------------	--	--

1 Summary

1.0.3 left a data layer in the folder that nothing loaded (1.0.3 \$5.5). 1.0.4 makes it the only source: the shop and the new product pages read `public/db/categorias.json` and `public/db/productos.json` through one loader in `js/tienda.jsx`. The inline `PRODUCTS`, `CATEGORIES` and `IMAGE_CREDITS` are gone; image credits now live in each product's `image` object. Each of the 30 products gets its own URL, `/pages/components/<slug>/`, generated by a Python script. The cart, product card and cart drawer move into `tienda.jsx` so the shop and every product page share them.

Staging. Prices, stock, specs and descriptions are sample data for the *MVP pre-launch manual input test*. Seller details (`SELLER.name`, `nif`, `address`) and both return addresses are still [...] placeholders, and these are shown on the terms page.

2 Files

File	Change
<code>public/db/productos.json</code> NEW	30 products, schema <code>lameyer.shop/v1</code> , <code>updated_at</code> 2026-10-03. 1017 lines.
<code>public/db/categorias.json</code> NEW	6 categories with <code>sort_order</code> , icon and description.
<code>public/db/README.md</code> NEW	Field reference for both “tables” and the manual input workflow (the file 1.0.3 \$5.5 found missing).
<code>js/tienda.jsx</code>	+548 lines, 651 in total. Data layer, cart, shared components and <code>ProductPage</code> / <code>renderProduct</code> .
<code>pages/components/index.html</code>	525 → 199 lines. Inline data and components removed; the shop now uses <code>useCatalog()</code> .
<code>pages/components/<slug>/index.html</code> ×30 NEW	Generated page shells, 39 lines each.
<code>tools/generar-paginas-producto.py</code> NEW	Page generator, 91 lines, standard library only. <code>tools/</code> is no longer empty.
<code>readme.md</code>	Folder tree, “Add a product” section, 1.0.4 entry.

3 How it works

3.1 Data layer

js/tienda.jsx · lines 105–145

```
const DB_BASE = '/public/db';
const loadJson = (name) =>
  fetch(`${DB_BASE}/${name}.json`, { cache: 'no-cache' }) // revalidate so manual edits show up at once
    .then(r => (r.ok ? r.json() : Promise.reject(new Error(`${name}.json: HTTP ${r.status}`))));

db.catalog() // { categories (sorted by sort_order), products (active !== false) }, fetched once per page
useCatalog() // { status: 'loading' | 'ready' | 'error', categories, products, byId, bySlug, category }
```

- **One seam for a real database.** Only `loadJson` / `db.catalog()` touch storage. Moving to Supabase, D1 or an API means replacing them and keeping the field names.
- **Loading and error states.** `CatalogStatus` shows three skeleton cards while loading and a “No se pudo cargar el catálogo” panel with a WhatsApp button on error.
- **Inactive products** (`active: false`) are dropped at load, so they vanish from the grid, cart and related items, and their page shows “Producto no encontrado”.

3.2 Catalogue contents (measured)

Measure	Value
Products	30, all active; 5 in each category (<code>oficina</code> , <code>estudiante</code> , <code>dev</code> , <code>redes</code> , <code>gamer</code> , <code>otros</code>)
Grades	16 <code>estandar</code> , 14 <code>premium</code>
Prices (EUR, VAT incl.)	9.90 – 89.90, average 33.80
Stock	One product at 0 (<code>bisagras-dell-latitude-5490-5480</code> , “Agotado”); the rest 4–50. None in the 1–3 “Últimas unidades” band, so that state is untested by the sample.
Images	29 with <code>image</code> ; <code>of-hinge-5490</code> is <code>null</code> and shows its category icon
Uniqueness	No duplicate <code>slug</code> or <code>sku</code>

3.3 Product pages

tools/generar-paginas-producto.py · js/tienda.jsx lines 435–651

The generator writes one static shell per active product. The shell contains only what crawlers and link previews need without JavaScript: `<title>` , meta description (description + price + “Devolución en 30 días”, max 300 chars), canonical URL, Open Graph tags with the 800 px photo, and a `<noscript>` heading. Then it calls `renderProduct()` , which finds the product by the last URL segment (`slugFromUrl()`) and renders:

- photo with credit, grade badge, stock status, price in the selected currency (EUR plus USD, MXN, GBP, ILS at ECB rates via Frankfurter, with fixed fallbacks);
- quantity selector (1 to stock, max 99), *Añadir al carrito* and *Comprar ahora* (adds and opens the cart);
- specs table with SKU, full `compatible_models` list, a WhatsApp “is it compatible?” link with the SKU prefilled, and a “notify me” link when out of stock;
- up to three related products from the same category, and links to returns, guarantee and shipping in the terms;
- schema.org `Product` / `Offer` JSON-LD (price, currency, availability, condition), injected client-side.

The script refuses to run on duplicate slugs and lists folders that no longer match an active product; it never deletes them. Price, stock and spec edits need no regeneration because pages read the JSON at load; slug, name, compat, description or image changes do.

3.4 Shop page

Category chips keep the selection in `?cat=<id>` . Search matches name, compat, every compatible model and the SKU. Each card links to its product page, and out-of-stock cards show “Agotado” with the button disabled. The first three cards load eagerly (unchanged from 1.0.3).

3.5 Cart and order

js/tienda.jsx · lines 161–188, 265–400

- **Storage:** `localStorage['lm-cart'] = { productId: qty }` , via a `store` wrapper that never throws (private mode, blocked storage).
- **Cross-tab sync:** a `storage` listener updates the cart when another tab changes it, e.g. a product page in one tab and the shop in another.
- **Stale ids:** lines whose product is missing or inactive are dropped once the catalogue loads.
- **Checkout:** name, delivery (home delivery needs a 5-digit postcode; pickup in Madrid does not) and acceptance of the terms. The order is sent as a prefilled WhatsApp message to `SELLER.whatsapp` :

```
Hola Lameyer, quiero hacer este pedido:
• 2 x Memoria RAM DDR5 ... (Portátiles y mini PC con DDR5) [SKU] – 179,80 €
Total: 179,80 € (IVA incluido) · ≈ <other currency, if selected>
Nombre: ...      Entrega: envío a domicilio · CP 28001
He leído y acepto los Términos y Condiciones: https://lameyer.net/pages/terminos/
```

Prices are charged in EUR; any other currency is shown as an approximation only.

4 Manual input test: adding or editing a product

1. Copy a product object in `productos.json` ; give it a new `id` , `slug` and `sku` . Keep the JSON valid and update `updated_at` .
2. Add `img/productos/<id>-400.webp` and `-800.webp` with credit fields, or set `"image": null` .
3. Run `python tools/generar-paginas-producto.py` from the repo root (or copy a product folder and edit its head by hand).
4. Check the shop grid, search, the product page, the cart and the WhatsApp message, then commit and push.

5 Known issues and limits

- **5.1 Cart ignores stock.** `useCart.add` and `setQty` cap at 99, not at `product.stock`. The product page limits one add to the stock, but adding twice, pressing “Añadir otro” on a card, or pressing `+` in the cart goes past it. A product that drops to 0 stock also stays in existing carts and can still be ordered.
- **5.2 Structured data is client-side only.** The `Product` JSON-LD exists only after React renders. Google executes JavaScript, but other crawlers and link-preview bots see only the static head (title, description, OG tags).
- **5.3 Generated heads can go stale.** Name, description and price in each shell's head are copied at generation time. A price edit shows correctly on the page but the meta description keeps the old price until the script is run again.
- **5.4 No data validation.** Nothing checks that `category_id` exists, that `grade` is `estandar` or `premium`, that prices are numbers, or that image files exist. A broken JSON file takes down the whole shop (error state), not just one product.
- **5.5 Public sample data.** `/public/db/` is served publicly by design (README warns against private data). Staging pages are live under the production domain and indexable, with sample prices and placeholder seller data.
- **5.6 Rates on every load.** Each page requests Frankfurter rates; nothing is cached beyond the browser's HTTP cache.

5.7 Legal placeholders. The terms page renders `[Razón social]`, `[NIF / CIF]`, `[Domicilio fiscal completo]` and two return addresses with `[Calle y número]`. Spanish e-commerce law (LSSI-CE) requires the seller's identity, tax ID and address. This blocks launch, not staging.

6 Status of the 1.0.3 gaps

#	1.0.3 gap	Status after 1.0.4	State
1	One catalogue source	<code>public/db/*.json</code> is the only source; inline arrays and <code>IMAGE_CREDITS</code> deleted; <code>public/db/README.md</code> written.	DONE
2	Schema validation	Only the generator's duplicate-slug check. See 5.4.	OPEN
3	Image script	Still manual. <code>tools/</code> now holds the page generator only.	OPEN
4	Git and tagged releases	The working copy is a clone of <code>github.com/eliav-mejia/lameyer.net</code> and 1.0.4 is commit <code>5d9647e</code> . There are still no tags, no <code>CHANGELOG.md</code> and no <code>package.json</code> .	PARTIAL
5	Unused stylesheets	Not addressed in this release.	OPEN
6	Real photos	Not addressed; stand-ins remain.	OPEN
—	<code>sitemap.xml</code> / <code>robots.txt</code>	Still missing, now with 30 more URLs to list.	OPEN

7 Before launch

#	Task	What it needs	Priority
1	Seller and return data	Replace every <code>[...]</code> in <code>SELLER</code> and <code>RETURN_ADDRESSES</code> (5.7).	BLOCKER
2	Real catalogue data	Check every price, stock figure, spec and compatible-model list by hand; set <code>updated_at</code> .	BLOCKER
3	Stock-aware cart	Clamp <code>add</code> / <code>setQty</code> to <code>product.stock</code> ; flag or remove lines whose stock fell to 0 before the WhatsApp message is built (5.1).	HIGH
4	Validation in the generator	Extend the Python script to check categories, grades, prices, image files and credits, and fail with a clear message (5.4).	HIGH
5	Static JSON-LD + sitemap	Have the generator also write the <code>Product</code> JSON-LD into each head and produce <code>sitemap.xml</code> and <code>robots.txt</code> (5.2, 5.3).	MED
6	Release tags	Tag <code>v1.0.1</code> – <code>v1.0.4</code> on the matching commits so releases can be diffed and rolled back.	MED

Items 4 and 5 fit in the existing `tools/generar-paginas-producto.py`: it already reads every product and writes every page, so validation, JSON-LD and the sitemap can be added there without a build system.